SENATI

Escuela de Tecnologías de la Información

GUÍA DE LABORATORIO

Guía 12

Visión Computacional y Machine Learning

Describe los principios de visión computacional y Machine Learning

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Curso	Seminario de Complementación Práctica I (PIAD-427)
Semana	12
Tarea de aprendizaje	Describe los principios de visión computacional y Machine Learning
Herramientas	Python 3 · Visual Studio Code · OpenCV · scikit-learn · TensorFlow
Instructor	Rembrandt Ubalde Enríquez
Estudiante	
Fecha	

Contenido

Objetivos de la guía 3
Materiales 3
Parte 1. Preparación del entorno en Visual Studio Code 4
1.1 Crear la carpeta del proyecto 4
1.2 Crear y activar el entorno virtual 4
1.3 Instalar las librerías 4
1.4 Seleccionar el intérprete y ejecutar scripts 4
Parte 2. Conceptos clave 6
2.1 ¿Qué es la visión computacional? 6
2.2 ¿Cómo «ve» una computadora? 6
2.3 Manejo de objetos: localización, clasificación y detección 6
2.4 Segmentación y reconocimiento de patrones 6
2.5 Principales aplicaciones 7
2.6 Librerías para leer imágenes desde la webcam (complementario) 7
Parte 3. Prácticas en Visual Studio Code 9
Práctica 01. Una imagen es una matriz 9
Práctica 02. Preprocesamiento de imágenes 10
Práctica 03. Localización, clasificación y detección de objetos 10
Práctica 04. Segmentación por color y con K-Means 12
Práctica 05. Machine Learning: clasificación de imágenes de dígitos 13
Práctica 06. Clasificación de imágenes con TensorFlow (complementario) 15
Práctica 07. Leer imágenes desde la webcam (complementario) 16
Parte 4. Errores frecuentes 18
Parte 5. Ejercicios propuestos 18
Preguntas de reflexión 18
Lista de cotejo 19

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Definir** la importancia de la visión computacional en la inteligencia artificial.
- **Describir** los fundamentos del manejo de objetos: localización, clasificación y detección.
- **Definir** la segmentación de imágenes y el reconocimiento de patrones.
- **Identificar** las principales aplicaciones de la visión computacional.
- **Clasificar** imágenes con Machine Learning y TensorFlow, y **leer** imágenes desde la webcam.

Operación	Dónde se trabaja en esta guía
Define la importancia de la Visión Computacional en IA	Parte 2 (secciones 2.1 y 2.2) y Práctica 01
Describe los fundamentos de la detección de objetos	Parte 2 (sección 2.3), Prácticas 02 y 03
Define la segmentación y reconocimiento de patrones	Parte 2 (sección 2.4) y Práctica 04
Identifica las principales aplicaciones	Parte 2 (sección 2.5)
Clasificación de imágenes con TensorFlow (complementario)	Prácticas 05 y 06
Librerías para leer imágenes desde webcam (complementario)	Parte 2 (sección 2.6) y Práctica 07

Materiales

- Computadora con Windows 10/11, Python 3.10 a 3.12 y Visual Studio Code con las extensiones **Python** y **Pylance**.
- **Cámara web** (integrada o USB) para la Práctica 07.
- Conexión a internet para instalar las librerías (solo la primera vez).

Parte 1. Preparación del entorno en Visual Studio Code

Si ya trabajaste las guías anteriores, tienes Python y VS Code listos. Igual sigue estos pasos para crear la **carpeta de esta guía** y su **entorno virtual**.

1.1 Crear la carpeta del proyecto

1. Crea en Documentos una carpeta llamada **IA_Guia12_Vision_Computacional**.
2. Abre Visual Studio Code y entra a **Archivo > Abrir carpeta…** (Ctrl + K, Ctrl + O). Selecciona la carpeta.
3. Si VS Code pregunta **«¿Confía en los autores de los archivos?»**, elige **Sí, confío en los autores**.
4. Abre la terminal integrada con **Terminal > Nuevo terminal** (Ctrl + ñ).

1.2 Crear y activar el entorno virtual

Terminal de VS Code

```
python -m venv venv
.\venv\Scripts\activate
```

Cuando el entorno está activo, la terminal muestra **(venv)** al inicio de la línea.

> ⚠ **Atención:** Si PowerShell muestra **«la ejecución de scripts está deshabilitada en este sistema»**, ejecuta una sola vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, responde **S** y vuelve a activar el entorno.

1.3 Instalar las librerías

Terminal de VS Code

```
pip install opencv-python numpy matplotlib scikit-learn tensorflow
```

> ⚠ **Atención:** Instala **opencv-python** y **no** `opencv-python-headless`. La versión «headless» no puede abrir ventanas y `cv2.imshow()` muestra el error **«The function is not implemented»**.

Verifica la instalación:

Terminal de VS Code

```
python -c "import cv2, sklearn; print(cv2.__version__, sklearn.__version__)"
```

▶ Resultado esperado en la terminal

```
4.x.x 1.x.x   (o versiones más recientes)
```

1.4 Seleccionar el intérprete y ejecutar scripts

1. Pulsa **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige el que dice **('venv': venv)**.

2. Crea cada archivo con el ícono **Nuevo archivo** del Explorador, con el nombre exacto que indica la guía (por ejemplo `01_imagen_matriz.py`).
3. Copia el código y guarda con **Ctrl + S**.
4. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo `python nombre_del_archivo.py` en la terminal.
5. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

> 💡 **Consejo:** Cuando un script abre una ventana (`plt.show()` o `cv2.imshow()`), el programa queda en pausa hasta que **cierres la ventana** o **presiones una tecla** sobre ella. Las imágenes también quedan guardadas como PNG en tu carpeta.

> 💡 **Consejo:** Para depurar paso a paso: haz clic a la izquierda del número de línea para poner un **punto de interrupción** (punto rojo) y pulsa **F5 > Python File**. En el panel **Variables** verás los valores de las matrices mientras el programa avanza.

Parte 2. Conceptos clave

2.1 ¿Qué es la visión computacional?

La **visión computacional** (o visión por computadora) es la rama de la IA que permite a las computadoras **ver, interpretar y entender** imágenes y videos. Su objetivo es que una máquina responda preguntas como: ¿qué hay en la imagen?, ¿dónde está?, ¿cuántos hay? y ¿qué está pasando?

Importancia en la IA: cerca del 80 % de la información que procesamos los humanos es visual. La visión computacional permite automatizar tareas que antes necesitaban una persona mirando: revisar productos en una línea de producción, leer placas, vigilar cámaras o analizar radiografías. Combinada con **Machine Learning** y **redes neuronales convolucionales (CNN)**, hoy supera a las personas en varias tareas de reconocimiento.

2.2 ¿Cómo «ve» una computadora?

Para la computadora una imagen es una **matriz de números**. Cada celda es un **píxel**:

Tipo de imagen	Forma de la matriz	Valores
Escala de grises	(alto, ancho)	0 = negro … 255 = blanco
Color	(alto, ancho, 3)	3 canales; OpenCV usa el orden **B, G, R** (azul, verde, rojo)
Video	Secuencia de imágenes (cuadros o *frames*)	Normalmente 30 cuadros por segundo

2.3 Manejo de objetos: localización, clasificación y detección

Tarea	Pregunta que responde	Resultado	Ejemplo
Clasificación	¿**Qué** hay en la imagen?	Una etiqueta para toda la imagen.	«Esta foto es un gato».
Localización	¿**Dónde** está el objeto?	Una **caja delimitadora** (x, y, ancho, alto).	Recuadro alrededor del gato.
Detección	¿**Qué** objetos hay y **dónde** está cada uno?	Varias cajas, cada una con su etiqueta.	3 autos y 2 personas en una calle.

Fundamentos de la detección: un sistema de detección (1) **preprocesa** la imagen (grises, desenfoque, bordes), (2) **propone regiones** donde puede haber objetos, (3) **clasifica** cada región y (4) dibuja las **cajas** con su etiqueta y su nivel de confianza. Los detectores modernos, como **YOLO** o **SSD**, hacen todo esto en un solo paso con una CNN.

2.4 Segmentación y reconocimiento de patrones

La **segmentación** divide la imagen en regiones, asignando a **cada píxel** una etiqueta. Es más precisa que una caja: indica el contorno exacto del objeto.

Tipo de segmentación	Qué hace	Ejemplo
Por umbral o color	Separa píxeles según su brillo o color.	Aislar las frutas rojas de una bandeja.
Por agrupamiento (K-Means)	Agrupa píxeles parecidos sin etiquetas.	Reducir una foto a 5 colores.
Semántica	Cada píxel recibe una clase (persona, calle, cielo).	Autos autónomos.
De instancias	Además separa cada objeto (persona 1, persona 2).	Contar personas en una fila.

El **reconocimiento de patrones** consiste en encontrar **regularidades** en los datos (formas, colores, texturas, trazos) y usarlas para tomar decisiones. Por ejemplo: un triángulo siempre tiene 3 vértices; el dígito «0» siempre tiene un trazo cerrado. Los algoritmos de Machine Learning aprenden estos patrones a partir de muchos ejemplos.

2.5 Principales aplicaciones

Sector	Aplicación
Seguridad	Reconocimiento facial, lectura de placas (ANPR), detección de intrusos.
Salud	Detección de tumores en radiografías y tomografías, análisis de lunares en la piel.
Industria	Control de calidad: detectar productos defectuosos en una faja transportadora.
Transporte	Autos autónomos: detectan peatones, semáforos y líneas de la pista.
Comercio	Tiendas sin cajeros, búsqueda de productos por foto, conteo de clientes.
Agricultura	Drones que detectan plagas o calculan la madurez de los cultivos.
Documentos	OCR: convertir fotos de documentos en texto editable.

2.6 Librerías para leer imágenes desde la webcam (complementario)

Librería	Tipo	Uso
OpenCV (`opencv-python`)	Escritorio	`cv2.VideoCapture(0)` lee la cámara; la más usada en visión computacional.
Tkinter + Pillow	Escritorio	Muestra el video de OpenCV dentro de una ventana con botones.
PyQt / PySide	Escritorio	Interfaces profesionales con video en vivo.
Streamlit (`st.camera_input`)	Web	Toma una foto desde el navegador en pocas líneas.
Gradio	Web	Crea una demo web de un modelo con entrada de webcam.
JavaScript (`getUserMedia`) + **TensorFlow.js**	Web	Accede a la cámara desde el navegador y ejecuta modelos sin servidor.

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Una imagen es una matriz

Crearás con OpenCV una imagen con cuatro figuras de colores y la guardarás como **figuras.png**. Este archivo se usa en las Prácticas 02, 03 y 04, así que **ejecuta esta práctica primero**.

Archivo: 01_imagen_matriz.py

```
# 01_imagen_matriz.py - Una imagen es una matriz de números (OpenCV + NumPy)
import cv2
import numpy as np

# Creamos una imagen de 300 x 400 píxeles con fondo blanco (B, G, R)
img = np.full((300, 400, 3), 255, dtype=np.uint8)
cv2.circle(img, (80, 90), 50, (0, 0, 255), -1)                          # círculo rojo
cv2.rectangle(img, (180, 40), (280, 140), (255, 0, 0), -1)              # cuadrado azul
triangulo = np.array([[330, 140], [390, 140], [360, 40]])
cv2.fillPoly(img, [triangulo], (0, 170, 0))                             # triángulo
verde
cv2.rectangle(img, (60, 190), (300, 260), (0, 220, 255), -1)            # rectángulo
amarillo
cv2.imwrite("figuras.png", img)                                         # se usa en las
prácticas siguientes

print("Forma de la matriz (alto, ancho, canales):", img.shape)
print("Tipo de dato:", img.dtype, "| valores de 0 a 255")
print("Total de píxeles:", img.shape[0] * img.shape[1])
print("Píxel (fila 90, col 80)  [B G R] =", img[90, 80], "-> rojo")
print("Píxel (fila 90, col 230) [B G R] =", img[90, 230], "-> azul")
print("Píxel (fila 10, col 10)  [B G R] =", img[10, 10], "-> blanco")

gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print("Imagen en grises:", gris.shape, "| un solo canal")

cv2.imshow("Figuras", img)          # abre una ventana
cv2.waitKey(0)                      # espera que presiones una tecla
cv2.destroyAllWindows()
```

▶ Resultado esperado en la terminal

```
Forma de la matriz (alto, ancho, canales): (300, 400, 3)
Tipo de dato: uint8 | valores de 0 a 255
Total de píxeles: 120000
Píxel (fila 90, col 80)  [B G R] = [  0   0 255] -> rojo
Píxel (fila 90, col 230) [B G R] = [255   0   0] -> azul
Píxel (fila 10, col 10)  [B G R] = [255 255 255] -> blanco
Imagen en grises: (300, 400) | un solo canal
```

Análisis: la imagen tiene 300 filas, 400 columnas y 3 canales. El rojo puro en OpenCV es **[0 0 255]** porque el orden es **B, G, R**. Al pasar a grises la matriz pierde la tercera dimensión.

> ⚠ **Atención:** Con `cv2.waitKey(0)` el programa espera hasta que **presiones una tecla con la ventana de la imagen seleccionada**. Si cierras la ventana con la X, en algunas PC la terminal queda esperando; en ese caso presiona Ctrl + C.

Práctica 02. Preprocesamiento de imágenes

Antes de detectar objetos se prepara la imagen. Aplicarás los cuatro pasos clásicos del preprocesamiento.

Archivo: 02_preprocesamiento.py

```
# 02_preprocesamiento.py - Grises, desenfoque, umbral y bordes
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("figuras.png")
if img is None:
    raise SystemExit("No se encontró figuras.png: ejecuta primero la Práctica 01")

gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)              # 1. escala de grises
suave = cv2.GaussianBlur(gris, (5, 5), 0)                  # 2. quita ruido
_, binaria = cv2.threshold(suave, 240, 255, cv2.THRESH_BINARY_INV)  # 3. objetos =
blanco
bordes = cv2.Canny(suave, 50, 150)                          # 4. detecta bordes

print("Píxeles de objeto (umbral):", cv2.countNonZero(binaria))
print("Píxeles de borde (Canny)  :", cv2.countNonZero(bordes))

titulos = ["Original", "Grises", "Umbral (binaria)", "Bordes (Canny)"]
imagenes = [cv2.cvtColor(img, cv2.COLOR_BGR2RGB), gris, binaria, bordes]
plt.figure(figsize=(10, 2.6))
for i in range(4):
    plt.subplot(1, 4, i + 1)
    plt.imshow(imagenes[i], cmap="gray")
    plt.title(titulos[i]); plt.axis("off")
plt.tight_layout()
plt.savefig("g12_preproceso.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ Resultado esperado en la terminal

```
Píxeles de objeto (umbral): 40046
Píxeles de borde (Canny)  : 1626
```

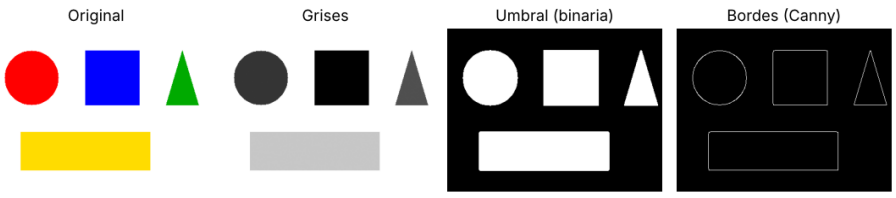


Figura 1. Original, grises, umbral (objetos en blanco) y bordes con Canny.

Práctica 03. Localización, clasificación y detección de objetos

Encontrarás cada figura (**localización** con una caja), reconocerás su forma contando vértices (**clasificación**) y dibujarás la caja con su nombre (**detección**).

Archivo: 03_localizar_clasificar.py

```
# 03_localizar_clasificar.py - Localización (dónde) y clasificación (qué) de objetos
import cv2

img = cv2.imread("figuras.png")
gris = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binaria = cv2.threshold(gris, 240, 255, cv2.THRESH_BINARY_INV)

# Contornos externos: cada contorno es un objeto
contornos, _ = cv2.findContours(binaria, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
print("Objetos encontrados:", len(contornos))

def clasificar(c):
    perimetro = cv2.arcLength(c, True)
    vertices = cv2.approxPolyDP(c, 0.03 * perimetro, True)   # simplifica el contorno
    n = len(vertices)
    if n == 3:
        return "triangulo"
    if n == 4:
        x, y, w, h = cv2.boundingRect(c)
        return "cuadrado" if 0.9 <= w / h <= 1.1 else "rectangulo"
    return "circulo"

for c in sorted(contornos, key=lambda c: cv2.boundingRect(c)[0]):
    x, y, w, h = cv2.boundingRect(c)                  # LOCALIZACIÓN: caja delimitadora
    forma = clasificar(c)                             # CLASIFICACIÓN: qué es
    print(f"{forma:<11} x={x:>3} y={y:>3} ancho={w:>3} alto={h:>3}
área={cv2.contourArea(c):>8.1f}")
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 0), 2)    # DETECCIÓN = dónde +
qué
    cv2.putText(img, forma, (x, y - 6), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 1)

cv2.imwrite("figuras_detectadas.png", img)
cv2.imshow("Deteccion de objetos", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

▶ Resultado esperado en la terminal

```
Objetos encontrados: 4
circulo     x= 30 y= 40 ancho=101 alto=101 área=  7704.0
rectangulo  x= 60 y=190 ancho=241 alto= 71 área= 16800.0
cuadrado    x=180 y= 40 ancho=101 alto=101 área= 10000.0
triangulo   x=330 y= 40 ancho= 61 alto=101 área=  3000.0
```

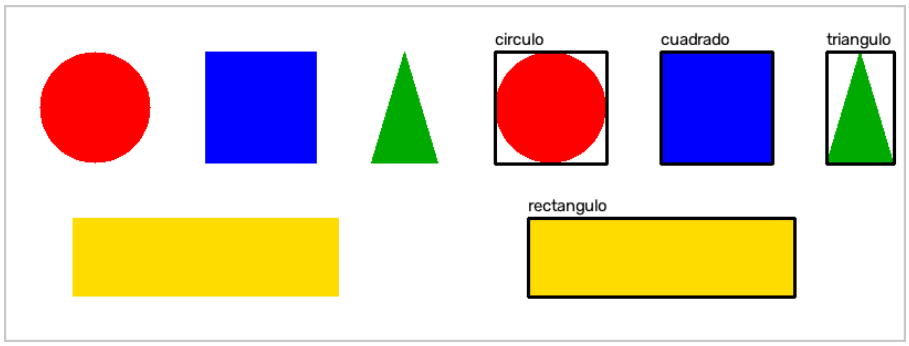


Figura 2. Imagen original (izquierda) y objetos detectados con su etiqueta (derecha).

Análisis: `findContours` encontró los 4 objetos. `approxPolyDP` simplifica cada contorno a sus vértices: 3 = triángulo, 4 = cuadrilátero (cuadrado si ancho ≈ alto), más de 4 = círculo. Este es un ejemplo sencillo de **reconocimiento de patrones** basado en reglas.

Práctica 04. Segmentación por color y con K-Means

Archivo: 04_segmentacion.py

```
# 04_segmentacion.py - Segmentación por color (HSV) y por agrupamiento (K-Means)
import cv2
import numpy as np

img = cv2.imread("figuras.png")

# A) SEGMENTACIÓN POR COLOR: aislar solo lo ROJO
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)              # H = tono, S = saturación, V
= brillo
mascara1 = cv2.inRange(hsv, (0, 120, 70), (10, 255, 255))      # el rojo está en ambos
extremos de H
mascara2 = cv2.inRange(hsv, (170, 120, 70), (180, 255, 255))
mascara_roja = mascara1 | mascara2
solo_rojo = cv2.bitwise_and(img, img, mask=mascara_roja)
print("Píxeles rojos:", cv2.countNonZero(mascara_roja))
cv2.imwrite("segmento_rojo.png", solo_rojo)

# B) SEGMENTACIÓN CON K-MEANS: agrupa los píxeles en k colores (patrones)
pixeles = img.reshape(-1, 3).astype(np.float32)         # 120 000 filas x 3 columnas
(B, G, R)
criterio = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 1.0)
cv2.setRNGSeed(0)
_, etiquetas, centros = cv2.kmeans(pixeles, 5, None, criterio, 5,
cv2.KMEANS_PP_CENTERS)
centros = centros.astype(np.uint8)
conteo = np.bincount(etiquetas.flatten())
print("\nColores descubiertos por K-Means (B, G, R) y % de la imagen:")
for c, n in sorted(zip(centros.tolist(), conteo), key=lambda t: -t[1]):
    print(f"  {str(c):<16} {n / len(pixeles):>6.1%}")

segmentada = centros[etiquetas.flatten()].reshape(img.shape)
cv2.imwrite("segmentada_kmeans.png", segmentada)
cv2.imshow("Solo rojo", solo_rojo)
cv2.imshow("K-Means", segmentada)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

▶ Resultado esperado en la terminal

```
Píxeles rojos: 7845

Colores descubiertos por K-Means (B, G, R) y % de la imagen:
  [255, 255, 255]   68.1%
  [0, 220, 255]     14.3%
  [255, 0, 0]        8.5%
  [0, 0, 255]        6.5%
  [0, 170, 0]        2.6%
```

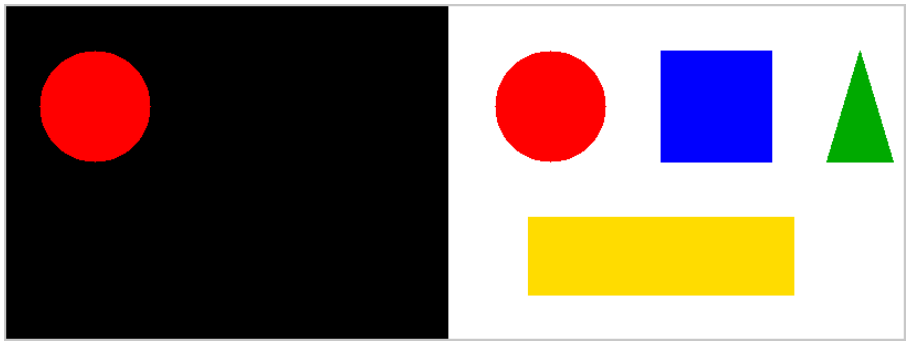


Figura 3. Segmentación del color rojo (izquierda) y segmentación en 5 colores con K-Means (derecha).

Análisis: en HSV el color depende solo del **tono (H)**, por eso es más fácil aislar un color que en BGR. K-Means (que estudiaste en la Guía 10) descubrió **sin etiquetas** los 5 colores de la imagen y cuánto ocupa cada uno: el fondo blanco es el 68 %.

> 💡 **Consejo:** Los rangos de HSV en OpenCV son: H de 0 a 179, S y V de 0 a 255. El rojo aparece en los **dos extremos** de H (0-10 y 170-179), por eso se usan dos máscaras.

Práctica 05. Machine Learning: clasificación de imágenes de dígitos

Usarás el dataset **digits** de scikit-learn: 1 797 imágenes de dígitos escritos a mano de 8 × 8 píxeles. Cada imagen se convierte en un vector de 64 números para entrenar un clasificador.

Archivo: 05_clasificar_digitos.py

```
# 05_clasificar_digitos.py - Machine Learning para clasificar imágenes de dígitos
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

digitos = load_digits()                      # 1 797 imágenes de 8 x 8 píxeles (0 a 16)
print("Imágenes:", digitos.images.shape, "| Etiquetas:", digitos.target.shape)
print("Imagen 0 (matriz 8x8) -> dígito", digitos.target[0])
print(digitos.images[0].astype(int))

X = digitos.images.reshape(len(digitos.images), -1)    # cada imagen -> vector de 64
valores
X_tr, X_te, y_tr, y_te = train_test_split(X, digitos.target, test_size=0.25,
random_state=42)

modelo = SVC(gamma=0.001)                    # clasificador de máquina de soporte
vectorial
modelo.fit(X_tr, y_tr)
pred = modelo.predict(X_te)
print(f"\nExactitud en prueba: {accuracy_score(y_te, pred):.2%}")
print("Matriz de confusión (filas = real, columnas = predicho):")
print(confusion_matrix(y_te, pred))

plt.figure(figsize=(8, 2.2))
for i in range(6):
    plt.subplot(1, 6, i + 1)
    plt.imshow(X_te[i].reshape(8, 8), cmap="gray_r")
    plt.title(f"Pred: {pred[i]}\nReal: {y_te[i]}", fontsize=9); plt.axis("off")
plt.tight_layout()
plt.savefig("g12_digitos.png", dpi=110, bbox_inches="tight")
plt.show()
```

▶ **Resultado esperado en la terminal**

```
Imágenes: (1797, 8, 8) | Etiquetas: (1797,)
Imagen 0 (matriz 8x8) -> dígito 0
[[ 0  0  5 13  9  1  0  0]
 [ 0  0 13 15 10 15  5  0]
 [ 0  3 15  2  0 11  8  0]
 [ 0  4 12  0  0  8  8  0]
 [ 0  5  8  0  0  9  8  0]
 [ 0  4 11  0  1 12  7  0]
 [ 0  2 14  5 10 12  0  0]
 [ 0  0  6 13 10  0  0  0]]

Exactitud en prueba: 98.89%
Matriz de confusión (filas = real, columnas = predicho):
[[43  0  0  0  0  0  0  0  0  0]
 [ 0 37  0  0  0  0  0  0  0  0]
 [ 0  0 38  0  0  0  0  0  0  0]
 [ 0  0  0 44  0  1  0  0  1  0]
 [ 0  0  0  0 55  0  0  0  0  0]
 [ 0  0  0  0  0 59  0  0  0  0]
 [ 0  0  0  0  0  0 45  0  0  0]
 [ 0  0  0  0  0  0  0 40  0  1]
 [ 0  0  0  0  0  0  0  0 38  0]
 [ 0  0  0  0  0  1  0  1  0 46]]
```

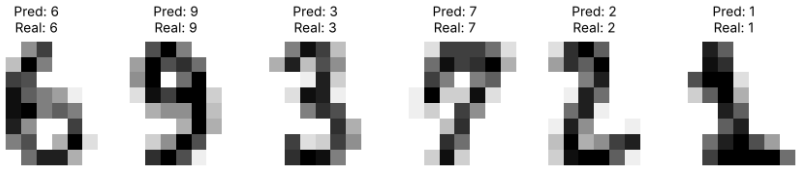


Figura 4. Algunas imágenes de prueba con su predicción y su etiqueta real.

Análisis: el modelo acertó el **98.89 %** de las 450 imágenes de prueba. En la matriz de confusión, la diagonal son los aciertos; por ejemplo, un «3» se confundió con un «5» y otro con un «8».

Práctica 06. Clasificación de imágenes con TensorFlow (complementario)

Ahora clasificarás los mismos dígitos con una **red neuronal convolucional (CNN)**, el tipo de red que estudiaste en la Guía 11. La CNN recibe la imagen como matriz **8 × 8 × 1**, sin aplanarla al inicio, y sus filtros aprenden los trazos.

Archivo: 06_cnn_tensorflow.py

```
# 06_cnn_tensorflow.py - Clasificación de imágenes con una CNN en TensorFlow
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

keras.utils.set_random_seed(42)
digitos = load_digits()
X = digitos.images[..., np.newaxis] / 16.0              # (1797, 8, 8, 1) normalizado
0 a 1
X_tr, X_te, y_tr, y_te = train_test_split(X, digitos.target, test_size=0.25,
random_state=42)

cnn = keras.Sequential([
    keras.Input(shape=(8, 8, 1)),
    layers.Conv2D(16, (3, 3), activation="relu"),       # 16 filtros detectan trazos
    layers.MaxPooling2D((2, 2)),                        # reduce el tamaño
    layers.Flatten(),
    layers.Dense(10, activation="softmax"),             # 10 clases: dígitos 0 a 9
])
cnn.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
print("Parámetros de la CNN:", cnn.count_params())

hist = cnn.fit(X_tr, y_tr, epochs=20, batch_size=32, validation_split=0.1, verbose=0)
print(f"Exactitud entrenamiento (última época): {hist.history['accuracy'][-1]:.2%}")
_, acc = cnn.evaluate(X_te, y_te, verbose=0)
print(f"Exactitud en prueba: {acc:.2%}")

prob = cnn.predict(X_te[:5], verbose=0)
print("Predichos:", prob.argmax(axis=1), "| Reales:", y_te[:5])
```

▶ Resultado aproximado (los decimales pueden variar en tu PC)

```
Parámetros de la CNN: 1610
Exactitud entrenamiento (última época): 97.53%
Exactitud en prueba: 96.89%
Predichos: [6 9 3 7 2] | Reales: [6 9 3 7 2]
```

Análisis: la CNN tiene solo **1 610 parámetros**: Conv2D (3·3·1·16 + 16 = 160) y Dense (3·3·16·10 + 10 = 1 450). Con 20 épocas alcanza alrededor del **97 %**. Con imágenes más grandes y complejas (fotos), las CNN superan ampliamente a los modelos clásicos.

Práctica 07. Leer imágenes desde la webcam (complementario)

Crearás una aplicación de escritorio que muestra la cámara en vivo y permite cambiar entre vista normal, en grises o de bordes, y guardar fotos.

Archivo: 07_webcam.py

```
# 07_webcam.py - Leer imágenes desde la webcam (aplicación de escritorio con OpenCV)
import cv2

camara = cv2.VideoCapture(0)                  # 0 = cámara principal de la laptop
if not camara.isOpened():
    raise SystemExit("No se pudo abrir la cámara. ¿Otra aplicación la está usando?")

modo = "normal"
print("Teclas: [n] normal  [g] grises  [b] bordes  [s] guardar foto  [q] salir")
fotos = 0
while True:
    ok, cuadro = camara.read()                # lee un cuadro (frame) del video
    if not ok:
        break
    cuadro = cv2.flip(cuadro, 1)              # efecto espejo
    if modo == "grises":
        vista = cv2.cvtColor(cuadro, cv2.COLOR_BGR2GRAY)
    elif modo == "bordes":
        vista = cv2.Canny(cv2.cvtColor(cuadro, cv2.COLOR_BGR2GRAY), 60, 140)
    else:
        vista = cuadro
    cv2.putText(vista, f"Modo: {modo}", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, 255,
2)
    cv2.imshow("Webcam - SENATI", vista)

    tecla = cv2.waitKey(1) & 0xFF             # espera 1 ms por una tecla
    if tecla == ord("n"): modo = "normal"
    elif tecla == ord("g"): modo = "grises"
    elif tecla == ord("b"): modo = "bordes"
    elif tecla == ord("s"):
        fotos += 1
        cv2.imwrite(f"foto_{fotos}.png", vista)
        print(f"Foto guardada: foto_{fotos}.png")
    elif tecla == ord("q"):
        break

camara.release()                              # libera la cámara
cv2.destroyAllWindows()
print("Programa terminado.")
```

▶ Resultado esperado (depende de las teclas que presiones)

```
Teclas: [n] normal  [g] grises  [b] bordes  [s] guardar foto  [q] salir
Foto guardada: foto_1.png
Foto guardada: foto_2.png
Programa terminado.
```

1. Ejecuta el script: se abre la ventana **«Webcam - SENATI»** con tu imagen en espejo.

2. Haz clic sobre la ventana del video (las teclas solo funcionan si la ventana está activa).
3. Presiona **g** para ver en grises, **b** para ver solo los bordes y **n** para volver a la vista normal.
4. Presiona **s** para guardar una foto: aparecerá `foto_1.png` en tu carpeta.
5. Presiona **q** para salir y liberar la cámara.

> ⚠ **Atención:** Si aparece «No se pudo abrir la cámara», cierra Zoom, Teams u otras apps que usen la cámara y revisa en **Configuración de Windows > Privacidad > Cámara** que las aplicaciones de escritorio tengan permiso. Si tienes una cámara USB adicional, prueba con `cv2.VideoCapture(1)`.

Parte 4. Errores frecuentes

Mensaje o síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'cv2'`	OpenCV no está instalado en el venv.	Activa el venv y ejecuta `pip install opencv-python`. El módulo se importa como `cv2`.
`cv2.error: … The function is not implemented`	Se instaló `opencv-python-headless`.	`pip uninstall opencv-python-headless` y luego `pip install opencv-python`.
`AttributeError: 'NoneType' object has no attribute 'shape'`	`cv2.imread` no encontró la imagen y devolvió `None`.	Ejecuta la Práctica 01 y verifica el nombre y la carpeta del archivo.
Los colores salen invertidos (rojo se ve azul) en Matplotlib	OpenCV usa BGR y Matplotlib usa RGB.	Convierte con `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.
La ventana de la imagen no responde	Falta `cv2.waitKey()` o la ventana no está activa.	Usa `cv2.waitKey(0)` y presiona una tecla con la ventana seleccionada.
«No se pudo abrir la cámara»	Otra app usa la cámara o no hay permiso.	Cierra otras apps y revisa los permisos de cámara de Windows.

Parte 5. Ejercicios propuestos

1. En la Práctica 02 cambia los umbrales de Canny de (50, 150) a (100, 200) y a (10, 50). ¿Cuántos píxeles de borde se detectan en cada caso? Explica qué hacen los dos umbrales.
2. Modifica la Práctica 04 para segmentar **solo los objetos azules** de `figuras.png` y dibuja una caja delimitadora alrededor de ellos.
3. Agrega un **pentágono** a la imagen de la Práctica 01 y mejora la función `clasificar` de la Práctica 03 para reconocerlo.
4. En la Práctica 05 compara la exactitud de **KNN**, **Árbol de decisión** y **SVM** con los mismos datos. ¿Cuál es el mejor clasificador de imágenes de dígitos?
5. Modifica la Práctica 07 para agregar el modo **«rojo»** (tecla r), que muestre solo los objetos rojos que aparecen frente a la cámara usando la máscara HSV de la Práctica 04.

Preguntas de reflexión

1. ¿Cuál es la diferencia entre clasificación, localización y detección de objetos? Da un ejemplo de cada una.
2. ¿Por qué la segmentación es más precisa que una caja delimitadora? ¿En qué aplicación sería indispensable?
3. ¿Por qué las CNN son mejores que un clasificador clásico cuando las imágenes son fotos grandes?
4. Menciona una aplicación de visión computacional que podría implementarse en tu centro de trabajo o en tu comunidad.

Lista de cotejo

- ☐ Instalé OpenCV, scikit-learn y TensorFlow en el entorno virtual.
- ☐ Explico la importancia de la visión computacional y cómo una computadora representa una imagen (Práctica 01).
- ☐ Apliqué grises, desenfoque, umbral y bordes (Práctica 02).
- ☐ Localicé, clasifiqué y detecté objetos con cajas delimitadoras (Práctica 03).
- ☐ Segmenté una imagen por color y con K-Means (Práctica 04).
- ☐ Clasifiqué imágenes con Machine Learning e interpreté la matriz de confusión (Práctica 05).
- ☐ Clasifiqué imágenes con una CNN de TensorFlow (Práctica 06).
- ☐ Leí imágenes desde la webcam con OpenCV (Práctica 07).
- ☐ Identifico al menos cinco aplicaciones de la visión computacional.
- ☐ Resolví al menos tres ejercicios propuestos.